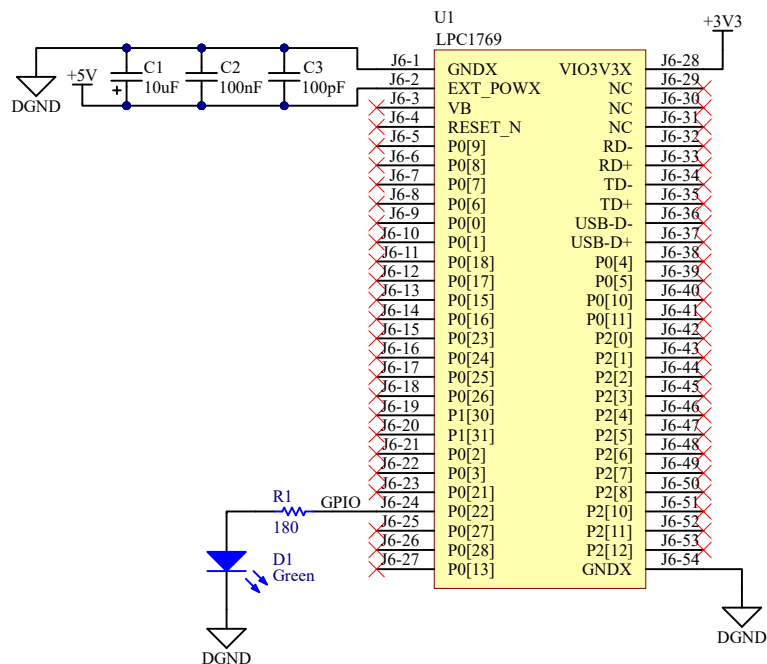


```
1  /*=====
2  * @file     Ejercicio_4_1.c
3  *
4  * @brief     Controla un LED conectado al pin P0.22, alternando periódicamente
5  *            entre los estados encendido y apagado, con un periodo de intermitencia
6  *            de 200ms.
7  *
8  * @details   Implementa la temporización mediante interrupciones periódicas
9  *            (no bloqueante), vía SysTick cada 100 ms.
10 *
11 * @author    Ing. E. Migliore
12 *
13 * @version   3.0
14  =====*/
```



```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5
6  /*=====
7  * DEFINICIÓN DE CONSTANTES
8  *=====*/
9  #define NVIC_PRIO_BITS      5U          /* Bits físicos de prioridad (0 al 31) */
10
11  /* Identificadores de Prioridad de IRQ */
12  #define SYSTICK_PRIORITY    0U
13
14  /* Identificadores de IRQ */
15  #define SYSTICK_IRQ         -1
16
17  /* Valores de recarga para SysTick */
18  #define N_TICKS_100MS      0x98967FU    // (100 ms * 100 MHz) - 1 = 9999999
19  // #define CCLK_FREQ_HZ      100000000U
20  // #define DELAY_100_MS      100U
21  // #define N_TICKS_S(cclk, t_s)    ((uint32_t)(((cclk) * (t_s)) - 1U))
22  // #define N_STICKS_MS(cclk, t_ms)  ((uint32_t)(((cclk) / 1000U) * (t_ms)) - 1U))
23  // #define N_STICKS_US(cclk, t_us)  ((uint32_t)(((cclk) / 1000000U) * (t_us)) - 1U))
24
25  #define SYSTICK_CTRL_ON     0x07U
26  #define SYSTICK_CTRL_OFF    0x00U
27
28  /*=====
29  * DEFINICIÓN DE VARIABLES GLOBALES
30  *=====*/
31
32  /*=====
33  * PROTOTIPOS DE FUNCIONES
34  *=====*/
35  static inline void cfgNVIC(int32_t, uint32_t);
36
37  void cfgGPIO(void);
38  void cfgSysTick(uint32_t, uint32_t);
39
40  /*=====
41  * FUNCIÓN PRINCIPAL
42  *=====*/
43  int main(void)
44  {
45      /*-----
46      * INICIALIZACIÓN DEL MCU
47      *-----*/
48      cfgGPIO();
49      cfgSysTick(N_TICKS_100MS, SYSTICK_CTRL_ON);
50      //cfgSysTick(N_STICKS_MS(DELAY_100_MS, CCLK_FREQ_HZ), SYSTICK_CTRL_ON);
51
52      /*-----
53      * INICIO PROGRAMA PRINCIPAL
54      *-----*/
55      while(1){}
56
57      return 0;
58  }
59
60  /*=====
61  * DEFINICIONES DE FUNCIONES
62  *=====*/
63  /*****
64  * @brief   Configura el NVIC y SCB para la habilitación de interrupciones.
65  *
66  * @param   <irq_number>      Número de IRQ en la tabla del vector.
67  * @param   <priority_number> Nivel de prioridad del vector.
68  *
69  * @return  None

```

```

70  *****/
71 static inline void cfgNVIC(int32_t irq_number, uint32_t priority_number)
72 {
73     //Asigna Prioridad de Interrupción
74     if(irq_number < 0) {
75         //Ejecuta interrupciones del sistema.
76         //SHP = System Handler Priority Registers
77         SCB->SHP[((uint32_t)(irq_number) & 0xF) - 4] = ((priority_number << (8 -
78             NVIC_PRIO_BITS)) & 0xFF);
79     } else {
80         //Ejecuta interrupciones de periféricos.
81         //IP = Interrupt Priority Registers
82         NVIC->IP[(uint32_t)(irq_number)] = ((priority_number << (8 - NVIC_PRIO_BITS)) &
83             0xFF);
84     }
85     //Habilitación de Interrupción
86     NVIC->ISER[((uint32_t)irq_number >> 5)] = (1 << ((uint32_t)irq_number & 0x1F));
87 }
88
89 /*****
90  * @brief   Configura las direcciones GPIO, modos de función y resistencias.
91  *
92  * @details - P0.22: Salida digital.
93  *
94  * @param   None
95  *
96  * @return  None
97  *****/
98 void cfgGPIO(void)
99 {
100     LPC_PINCON->PINSEL1 &= ~(0b11 << 12); //P0.22 con Funcionalidad GPIO
101     LPC_GPIO0->FIODIR |= (1<<22);          //P0.22 como Salida Digital
102
103     return;
104 }
105
106 /*****
107  * @brief   Configura el módulo SysTick Timer
108  *
109  * @param   <nTicks> Valor de recarga del registro LOAD.
110  * @param   <cTicks> Registro de control CTRL (Habilitación/Reloj/IRQ).
111  *
112  * @return  None
113  *****/
114 void cfgSysTick(uint32_t nTicks, uint32_t cTicks)
115 {
116     SysTick->LOAD = nTicks;          //Carga cuentas en STRELOAD
117     SysTick->VAL = 0;                 //Carga cuentas a cero STCURR
118     SysTick->CTRL = cTicks;           /* Habilita Contador (ENABLE) (1<<0)
119                                     * Habilita Interrupcion (TICKINT) (1<<1)
120                                     * Selecciona fuente de reloj (CLKSOURCE) (1<<2)
121                                     */
122
123     cfgNVIC(SysTick_IRQn, SYSTICK_PRIORITY);
124
125     return;
126 }
127
128 /*****
129  * @brief   Rutina de Servicio de Interrupción (ISR) para el vector SysTick.
130  *
131  * @details  Alterna el estado lógico del pin P0.22 en cada desborde (100 ms)
132  *           mediante los registros FIOSET y FIOCLR, logrando una onda cuadrada
133  *           con período total de 200 ms.
134  *
135  * @param   None
136  *****/

```

```
137     * @return    None
138     *****/
139 void SysTick_Handler(void)
140 {
141     static uint8_t vTicks = 0;
142
143     vTicks++;
144     if(vTicks%2){
145         LPC_GPIO0->FIOSET = (1<<22);
146     }
147     else{
148         LPC_GPIO0->FIOCLR = (1<<22);
149     }
150
151     return;
152 }
153
```